GPU MEMORY ARCHITECTURE DESIGN

Contents

Introduction 2

VRAM Category-based Memory Systems 3

GPU Data Operations and Processing Systems 4

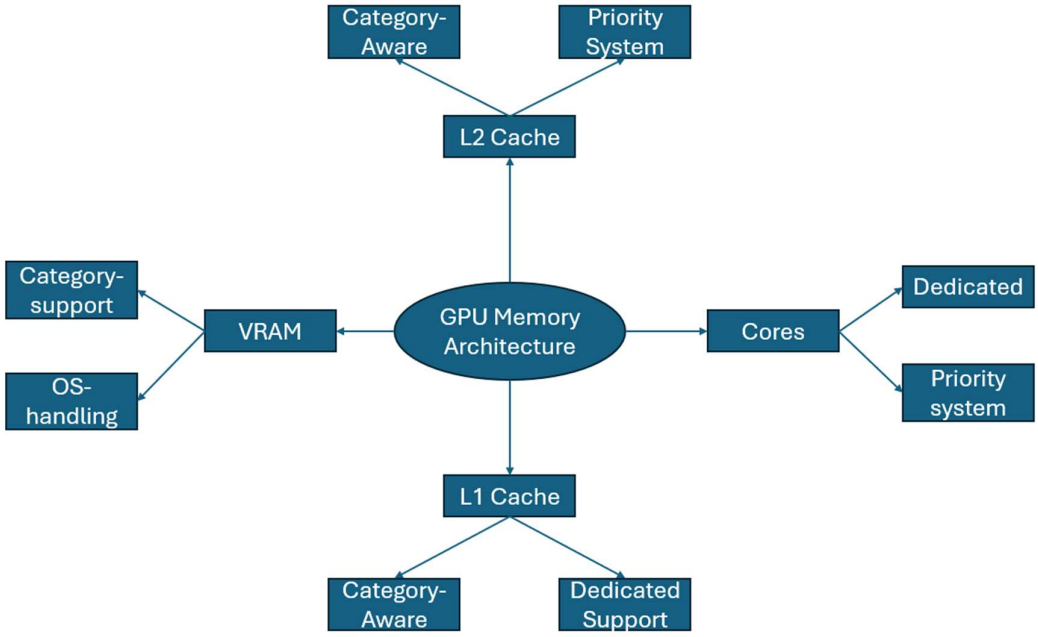
GPU Memory Architecture Workflow 5

GPU Memory Architecture Trade-offs 6

Introduction

This GPU Memory Architecture is designed to improve performance of GPUs in modern computing systems. This was also developed to address the current limitations of GPU components such as VRAM and Cache. This GPU memory architecture was developed as current GPUs aren't designed to handle the heavy workloads that modern systems especially AI and gaming generate. This GPU memory architecture design will explain the changes made to VRAM, Cache and Cores. It will explain the workflow and the trade-offs of the design.

This GPU Memory Architecture cannot promise perfect performance or handling of workloads. However, it can promise to improve performance and handling of workloads compared to current GPU memory architecture. It is important to remember that this is just a design, and engineers may face more challenges than defined within this specification document. This GPU Memory architecture should also be adapted based on the system that it is used for. For example, gaming systems may prioritise apps over the operating system.



The spider diagram shows a visual representation of the GPU memory architecture. It shows two features of all 4 components involved in the architecture. This spider diagram does not show the exact details of how these features or systems work. More detail on the architecture can be found in the sections below.

VRAM Category-based Memory Systems

- **This VRAM category-based memory system is designed to organise the data that is stored within VRAM efficiently.**
- **There are 3 subsystems involved in the category-based memory system.**
- **The first system is the space allocation system.**
- **VRAM will be sorted into 4 categories.**
- **The first category is frame buffers which stores data such as colour data, alpha channel and depth buffers.**
- **The second category is models which stores data such as physics models, 3D geometry, and skeletal data.**
- **The third category is textures which stores data such as colour maps, normal maps, displacement maps, and roughness maps.**
- **The fourth and final category is free space which stores no data and can be used as secondary storage for the other categories.**
- **For frame buffers, the storage for the category will be decided by using screen resolution plus a 10% safety buffer to ensure that it has enough space.**
- **For models and textures, each category will each be given 20% of what remains of the space after frame buffers has been calculated.**
- **For the free space category, the remaining space after all 3 main categories has been calculated will be allocated to it.**
- **The second system is VRAM fallback system.**
- **This system has 2 layers.**
- **The first layer is the 3 main categories using storage that is stored within the free space category.**
- **This means that categories won't run out of space while not using up space that the other categories are currently using.**
- **Once a main category doesn't need the space, the free space category gets the space back and is ready for usage again.**
- **The second layer is the fallback to System RAM if the free space category runs out of space in real-time.**
- **This means that one of the main categories can use space available in the app category of system RAM for storage.**
- **It is important that it doesn't become a long-term action as it could starve app processes in RAM from storage and processing.**
- **The third and final subsystem is OS sorting for the categories.**
- **The operating system will be the main factor in ensuring that data is sorted into categories within VRAM.**
- **The operating's system memory manager will have a list of defined data that goes into each category as well as probabilistic analysis for undefined data.**
- **This probabilistic analysis system works by looking at what the data is being used for and using heuristics to determine what category it should be in.**

GPU Data Operations and Processing Systems

- **These GPU data operations and processing systems are designed to make the GPU use the category-based memory structure efficiently.**
- **This massive system has 4 main subsystems.**
- **The first subsystem is the hybrid priority system for L2 cache space allocation.**
- **This system defines what categories get the most space and how this priority-based allocation works.**
- **For frame buffers, the category is given a fixed space within in L2 cache which is about 50% of cache space in total.**
- **For textures and models, both categories are given 25% of the remaining space left within the cache.**
- **However, the OS or a scheduling algorithm within the GPU can allocate more storage based on which ever of the one category is more demanding.**
- **The most demanding would be decided based on what is storing the most data within VRAM and is running the most intensive processing.**
- **The second subsystem is the hybrid priority system for SM/CUs and their L1 caches for each of them.**
- **This system defines what category each core gets and how L1 caches are allocated in real-time during active operations.**
- **For frame buffers, 40-50% of the total GPU cores will be allocated to those categories to be able run heavy loads such as ultra 4K and ray tracing.**
- **For textures and models, the remaining 50% will be split into 25% each while it may be split differently depending on how much space frame buffers takes.**
- **The OS or a scheduling algorithm within the CPU can dynamically change the number of cores that models and textures use in real-time based on workload.**
- **This allocation is decided based on the storage used in VRAM, the performance of the application and if tasks for the category need more power.**
- **The third subsystem is the write system for the GPU which defines how data is written back from cores to VRAM.**
- **This subsystem defines the writing that the GPU needs to do to store data back where it can remain long-term.**
- **This system starts off with the cores writing results directly to the registers that are implemented within them.**
- **From the registers, the GPU's memory controller writes the results from the cores directly to VRAM.**
- **This is done by the VRAM bus which still pass by the caches but doesn't go through them in the sense that no data is written to them.**
- **This system was chosen as it makes writing more efficient and quicker allowing for the GPU to process and use results more quickly.**
- **The fourth and final system is the read system.**
- **The GPU fetches from VRAM, which the bus puts the data into L2 and L1 caches and then the cores use this data to execute tasks.**

GPU Memory Architecture Workflow

- **This workflow will help anyone trying to create this design to understand how the process works.**
- **It is also important to know that this workflow uses the dynamic priority system workflow instead of fixed.**
- **First, data is loaded from RAM by the CPU into VRAM.**
- **The OS organises the data loaded from RAM into the categories models, textures and frame buffers.**
- **For example, the user is playing GTA 6 which demands heavy physics workloads.**
- **Second, the most frequently used data of the models category is placed into L2 cache by the priority system.**
- **The other two categories, textures and frame buffers are allocated less L2 cache space by the priority system.**
- **Third, the most frequently used data of each category is separated into the dedicated L1 cache of each SM/CU.**
- **This is done by the priority system as most L1 caches will be dedicated to the models category.**
- **Fourth, each SM/CU is dedicated to a specific category by the priority system based on the heaviest workload.**
- **This means that most SM/CU would be dedicated to the models category as it would be the heaviest workload produced by GTA 6.**
- **Fifth, data is loaded into registers per thread for SM/CU to access easily without heading to cache.**
- **The SMs/CUs would then execute the instructions or tasks needed using the data stored in registers, cache and even VRAM if needed.**
- **Sixth, results would be written directly to VRAM after they are produced.**
- **This means that from the registers the GPU writes the results directly to VRAM instead of writing through each cache.**
- **This is done by the GPU VRAM bus which directly writes the results to VRAM instead of passing through multiple caches.**
- **Finally, the results are displayed on the user's interface of GTA 6.**
- **This means that the user will be able to see the physics models and other assets in the game once the results have been delivered and processed.**
- **Make sure for certain systems such as consoles or embedded systems this workflow is adapted for the fixed priority system.**

GPU Memory Architecture Trade-offs

- **This GPU Memory Architecture has trade-offs just like any other system I have created.**
- **I have identified 4 main trade-offs with this GPU Memory Architecture.**
- **Engineers may also face additional challenges during actual implementation of this architecture as I cannot test implementation trade-offs.**
- **The first trade-off is Heat.**
- **This trade-off exists because the GPU can do more work more quickly which can produce heat.**
- **This can cause the GPU to overheat especially in hot environments and long device usage hours.**
- **This trade-off can be managed through cooling systems such as heat sinks and fans to manage the heat.**
- **The second trade-off is Latency.**
- **This trade-off exists because of the priority system real-time changes to multiple components in the GPU.**
- **This means that the user may experience a lag spike as SMs/CUs, and cache must change categories and tasks even if it is a quick process.**
- **This trade-off can be managed through using a fixed priority system where possible and using a combination of hardware-level and OS-level management.**
- **The third trade-off is Time-Consumption.**
- **This trade-off exists because creating a new GPU design using this architecture can take years even if there are no direct hardware changes.**
- **This is because OS vendors and hardware engineers would need to coordinate for this GPU Memory architecture to exist.**
- **It is also because this GPU memory architecture may have to be adapted to multiple different systems.**
- **This trade-off can be managed through structured development processes and business agreements.**
- **It is also acceptable as new generations of GPU usually can be time-consuming to create in general.**
- **The fourth trade-off is OS-Reliance**
- **This trade-off exists because the OS must manage the sorting of data into VRAM categories and the real-time changes with the priority system.**
- **This means that the memory organisation of VRAM and the effectiveness of the priority system can be limited by OS limitations.**
- **This trade-off can be managed through hardware-level scheduling and pre-existing GPU systems that don't rely on the OS.**
- **This trade-off is already acceptable as operating systems manage VRAM and priority systems in modern computing systems.**